CM2 : Tests d'Intégration avec FastAPI

TestClient · Fixtures · Faker · JUnit XML · Smoke Tests · ShopFlow

Plan du cours

01 **Unitaire vs Intégration**

Pyramide des tests · Quand intégrer

02 **TestClient FastAPI**

httpx · Anatomie · Codes HTTP

03 **Fixtures SQLite intégration**

dependency_overrides · scope · teardown

04 **Scénarios ShopFlow**

Produit → Panier → Commande · Erreurs

05 **Données de Test & Faker**

Fixtures API · factory · données réalistes

06 **JUnit XML & Smoke Tests**

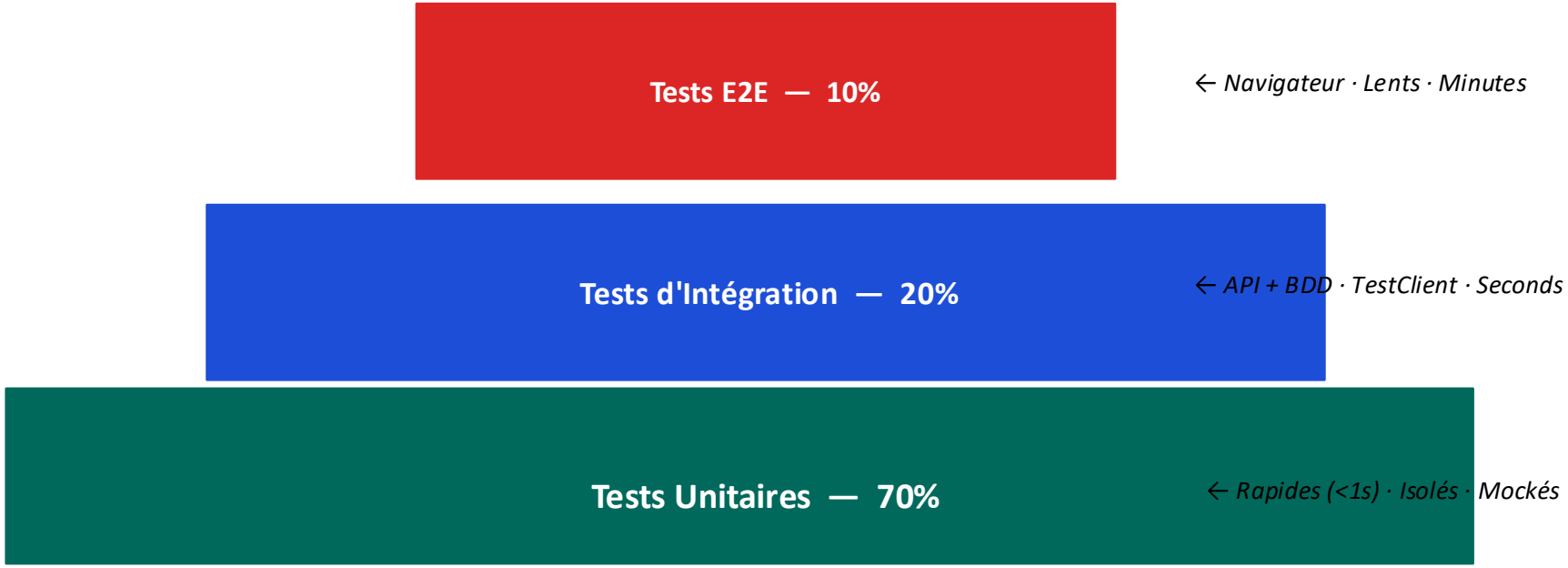
--junitxml · /health · pipeline Jenkins

P A R T I E 0 1

Unitaire vs Intégration

Pyramide des tests · Quand écrire des tests d'intégration

La Pyramide des Tests



Règle : plus on monte, plus c'est lent et fragile → favoriser la base de la pyramide

Unitaire vs Intégration : La différence concrète

Test UNITAIRE

```
# Teste calcul_prix_ttc() SEUL
def test_prix_normal():
    result = calcul_prix_ttc(100.0)
    assert result == 120.0

# Pas de BDD
# Pas de HTTP
# < 0.01s
```

- Teste UNE fonction isolée
- Services mockés
- Rapide : < 10ms
- Localise le bug précisément

Test INTÉGRATION

```
# Teste POST /products/ complet
def test_create_product(client):
    response = client.post(
        '/products/',
        json={'name':'PC','price':999})
    assert response.status_code == 201
    assert response.json()['id'] != None
```

- Teste route + service + BDD
- TestClient FastAPI (httpx)
- Plus lent : 50-200ms
- Valide le contrat HTTP

P A R T I E 0 2

TestClient FastAPI

httpx · Anatomie · Status codes · Sans serveur HTTP

TestClient : Requêtes HTTP sans serveur réel

TestClient (basé sur httpx) simule un vrai client HTTP directement en mémoire pas besoin d'uvicorn.

```
from fastapi.testclient import TestClient
from app.main import app

# Instanciation pas besoin d'uvicorn démarré
client = TestClient(app)

# Toutes les méthodes HTTP disponibles
client.get('/products/')                        # GET
client.post('/products/', json={...})           # POST
client.put('/products/1', json={...})           # PUT
client.delete('/products/1')                    # DELETE
client.patch('/orders/1/status', json={...})    # PATCH

# Avec paramètres de requête
client.get('/products/?category=info&limit=5')  # Query params
client.get('/products/', params={'category':'info'}) # Équivalent
```

Avantages TestClient

- Pas de port réseau utilisé
- Même processus Python → rapide
- Gère les middlewares FastAPI
- Lifespan (startup/shutdown) exécuté
- Compatible avec pytest fixtures

Dépendance requise

- pip install httpx
- httpx==0.27.0

Anatomie d'un test d'intégration complet

Un test d'intégration vérifie 3 choses : le status code HTTP, le contenu JSON, et les effets en BDD.

```python
@pytest.mark.integration
def test_create_product(client):
    # 1. ARRANGE préparer le body de la requête
    payload = {
        'name': 'Laptop Pro',
        'price': 999.99,
        'stock': 10
    }

    # 2. ACT envoyer la requête HTTP
    response = client.post('/products/', json=payload)

    # 3. ASSERT vérifier status + body + BDD
    assert response.status_code == 201          # HTTP Created
    data = response.json()
    assert data['name'] == 'Laptop Pro'         # données correctes
    assert data['id'] is not None               # id auto-généré
    assert 'created_at' in data                 # champ présent
```

Codes HTTP ShopFlow

200 OK	GET réussi
201 Created	POST réussi
204 No Content	DELETE réussi
400 Bad Request	Logique métier
404 Not Found	Ressource absente
422 Unprocessable	Validation Pydantic
409 Conflict	Duplicate (coupon)

P A R T I E 0 3

Fixtures SQLite pour l'Intégration

dependency_overrides · scope · teardown automatique

dependency_overrides : Injecter la BDD de test

FastAPI permet de remplacer n'importe quelle dépendance (Depends) par une version de test, sans modifier le code de production.

```python
# tests/conftest.py
@pytest.fixture(scope='module')
def client(db_engine):
    """TestClient FastAPI avec BDD SQLite isolée."""
    SessionTest = sessionmaker(bind=db_engine)

    def override_get_db():   # ← remplace get_db de production
        session = SessionTest()
        try:
            yield session
        finally:
            session.close()

    # Injecter la BDD de test dans FastAPI
    app.dependency_overrides[get_db] = override_get_db

    with TestClient(app) as c:
        yield c   # ← les tests utilisent ce client

    # Teardown : restaurer les dépendances originales
    app.dependency_overrides.clear()
```

Comment ça marche ?

1. Route appelle Depends(get_db)
2. FastAPI cherche dans dependency_overrides
3. override_get_db trouvé → SQLite test
4. Test s'exécute avec BDD isolée
5. clear() restaure get_db original

Scope des fixtures d'intégration : Module vs Function

Le choix du scope impacte la vitesse ET l'isolation des tests.

scope='module' est RECOMMANDÉ

```
@pytest.fixture(scope='module')
def client(db_engine):
    # Client créé UNE FOIS
    # pour tout test_products_api.py
    ...
    with TestClient(app) as c:
        yield c
```

Client créé 1× par fichier
Lifespan exécuté 1×
BDD partagée entre tests

scope='function' consiste en une ISOLATION TOTALE

```
@pytest.fixture(scope='function')
def client(db_engine):
    # Client recréé pour
    # CHAQUE test individuel
    ...
    with TestClient(app) as c:
        yield c
```

BDD vierge par test
Isolation parfaite
Plus lent (×N lifespan)

P A R T I E 0 4

Scénarios d'Intégration ShopFlow

Produit → Panier → Commande · Tester les erreurs

Scénario complet — Produit → Panier → Commande

Le test d'intégration le plus important de ShopFlow : valider le flux e-commerce de bout en bout.

```
def test_flux_complet_commande(client):
    # ÉTAPE 1 : Créer un produit (stock=10)
    p = client.post('/products/',
        json={'name':'Laptop','price':999.99,'stock':10}).json()
    assert p['id'] is not None

    # ÉTAPE 2 : Ajouter au panier (user_id=42, qty=2)
    client.post(f'/cart/?user_id=42',
        json={'product_id': p['id'], 'quantity': 2})

    # ÉTAPE 3 : Passer la commande
    order = client.post('/orders/',
        json={'user_id': 42}).json()
    assert order['status'] == 'pending'
    assert order['total_ttc'] == 2399.98  # 999.99×2×1.20

    # ÉTAPE 4 : Vérifier effets de bord
    stock = client.get(f'/products/{p["id"]}').json()['stock']
    assert stock == 8        # 10 - 2 = 8
    cart = client.get('/cart/42').json()['items']
    assert cart == []        # panier vidé après commande
```

Tester les cas d'erreur — 400, 404, 422

Tester les erreurs est aussi important que tester le cas nominal, elles définissent le contrat de l'API.

```
# 422 : Validation Pydantic
def test_prix_negatif_rejete(client):
    r = client.post('/products/', json={'name':'X','price':-5.0})
    assert r.status_code == 422   # Pydantic Field(gt=0)
    assert 'detail' in r.json()   # message d'erreur présent

# 404 : Ressource inexistante
def test_produit_inexistant(client):
    r = client.get('/products/99999')
    assert r.status_code == 404

# 400 : Logique métier
def test_commande_panier_vide(client):
    r = client.post('/orders/', json={'user_id': 9999})
    assert r.status_code == 400   # panier inexistant
    assert 'vide' in r.json()['detail'].lower()

# 409 : Conflit (doublon)
def test_coupon_duplique(client):
    client.post('/coupons/', json={'code':'PROMO','reduction':10})
    r = client.post('/coupons/', json={'code':'PROMO','reduction':20})
    assert r.status_code == 409
```

P A R T I E 0 5

Données de Test & Faker

Fixtures API · Factory pattern · Données réalistes

Fixtures API : Données créées via l'API

Les fixtures API créent les données via les endpoints REST
Elles testent aussi la création en passant.

```python
# tests/conftest.py - fixtures données via API

@pytest.fixture
def api_product(client):
    """Crée un produit via POST /products/ et le retourne."""
    response = client.post('/products/', json={
        'name': 'Clavier Mécanique',
        'price': 89.99,
        'stock': 25,
        'category': 'peripheriques'
    })
    assert response.status_code == 201  # vérifie la création
    yield response.json()               # retourne le JSON complet
    # Cleanup automatique via soft delete
    client.delete(f'/products/{response.json()["id"]}')

# Utilisation dans un test :
def test_get_product(client, api_product):
    r = client.get(f'/products/{api_product["id"]}')
    assert r.status_code == 200
    assert r.json()['name'] == api_product['name']
```

Faker : Générer des données réalistes

Faker génère automatiquement des noms, prix, emails realists, évite les données statiques qui se dupliquent.

```
from faker import Faker
fake = Faker('fr_FR')   # données en français

# Générer des données produit réalistes
def make_product_data():
    return {
        'name'     : fake.word().capitalize()+' Pro',
        'price'    : round(fake.pyfloat(min_value=5, max_value=2000, right_digits=2), 2),
        'stock'    : fake.random_int(min=1, max=100),
        'category' : fake.random_element(['informatique','peripheriques','audio']),
    }

# Fixture factory crée N produits différents
@pytest.fixture
def make_product(client):
    """Factory : retourne une fonction qui crée un produit."""
    def _make(**kwargs):
        data = {**make_product_data(), **kwargs}  # merge
        r = client.post('/products/', json=data)
        return r.json()
    return _make

# Utilisation flexible :
def test_pagination(client, make_product):
    for _ in range(10): make_product()          # 10 produits random
    make_product(price=9.99, category='promo')  # surcharge possible
```

Faker : Pourquoi éviter les données statiques ?

Données statiques

```
# Même code pour 5 tests
{'name': 'Laptop',
 'price': 999.99,
 'stock': 10}
# UNIQUE constraint si scope=module!
```

Faker : Données uniques

```
# Chaque test = données uniques
{'name': 'Clavier Pro',
 'price': 247.83,
 'stock': 37}
# Jamais de collision !
```

Générateurs Faker utiles pour ShopFlow

`fake.word()`	Nom de produit aléatoire
`fake.random_int(min=1, max=100)`	Quantité / stock
`fake.bothify('PROMO##')`	Code coupon unique (ex: PROMO47)
`fake.pyfloat(min=5, max=2000)`	Prix réaliste
`fake.email()`	Email utilisateur
`fake.random_element([...])`	Catégorie aléatoire dans une liste

P A R T I E 0 6

JUnit XML & Smoke Tests

--junitxml · GET /health · Pipeline Jenkins

Rapport JUnit XML : Publication dans Jenkins

--junitxml génère un fichier XML standard que Jenkins publie dans son interface pour visualiser les résultats.

```
# Générer le rapport JUnit XML
pytest tests/integration/ \
    -v \
    --junitxml=junit-integration.xml \
    --no-cov

# Structure du fichier généré :
# junit-integration.xml
<?xml version='1.0' encoding='utf-8'?>
<testsuites>
  <testsuite name='integration' tests='16'>
    <testcase
      classname='test_products_api.TestCreate'
      name='test_creation_valide'
      time='0.123'/>
    <testcase name='test_prix_negatif'>
      <failure message='assert 200 == 422'/>
    </testcase>
  </testsuite>
</testsuites>
```

Dans le Jenkinsfile (TP3)

```
stage('Integration Tests') {
  steps {
    sh '''
      pytest tests/integration/ \
        --junitxml=junit-int.xml \
        --no-cov
    '''
  }
  post { always {
    junit 'junit-int.xml'
  }}
}
```

Ce que Jenkins affiche

- Nombre total de tests
- Tests passés / échoués / ignorés
- Durée par test
- Graphique de tendance (builds)
- Détail des erreurs en 1 clic

Smoke Tests : Vérifier que l'API est vivante

Les smoke tests sont les premiers tests lancés dans le pipeline CI : en 2 secondes, on sait si l'API démarre correctement.

```python
# tests/integration/test_health.py
"""
Smoke tests — premier stage Jenkins.
Si ces tests échouent, l'API ne démarre pas → tout le reste est inutile.
"""
import pytest

@pytest.mark.integration
class TestHealth:

    def test_health_ok(self, client):
        """GET /health → {status: ok} — smoke test principal."""
        r = client.get('/health')
        assert r.status_code == 200
        assert r.json()['status'] == 'ok'
        assert 'version' in r.json()

    def test_docs_accessible(self, client):
        """Swagger UI doit être accessible."""
        assert client.get('/docs').status_code == 200

    def test_root_accessible(self, client):
        assert client.get('/').status_code == 200
```

Smoke Tests en CI

- Stage 1 du pipeline Jenkins
- Si échoue → tout le pipeline s'arrête
- Détecte : imports cassés, BDD inaccessible,
- port déjà utilisé, variable d'env manquante
- Durée : quelques secondes

GET /health : Contrat

- status: 'ok'
- version: '0.1.0'
- db: 'sqlite'
- → Utilisé par Docker HEALTHCHECK
- → Monitoring en production